

CHANNEL Z0

DOC NO. RL-Z0-A REV. A RIPOSTE LABORATORIES

CONTENTS

▶ How it works	2
▶ Repository map	2
Station scripts	4
▶ Quickstart	6
▶ Configuration, all of it	7
The live chat in the rail	7
▶ The sponsor	8
▶ License	9
▶ Brand	9

CHANNEL Z0 · CH 0 · DESIG RL-Z0

a local channel, for locals

a Riposte Laboratories transmission

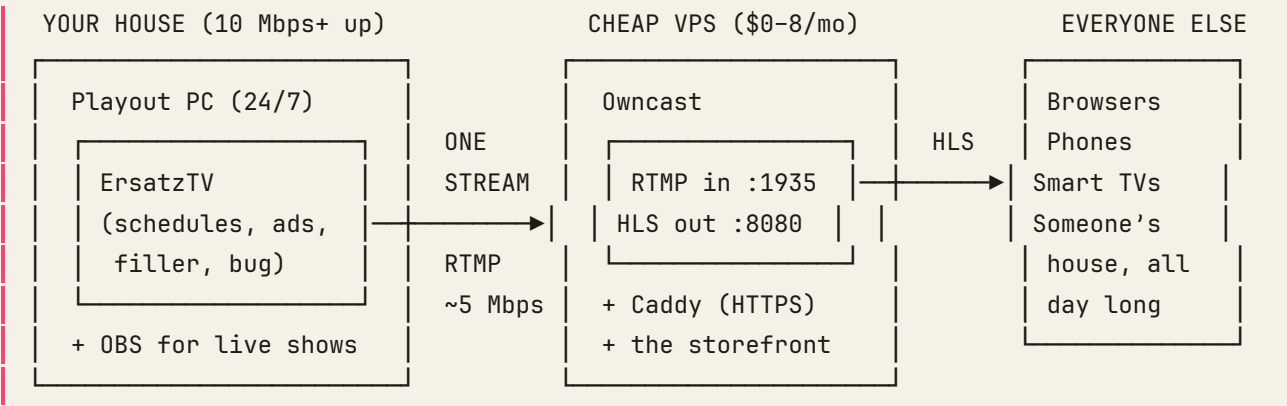
An old-style TV channel built on new plumbing: one linear channel that runs 24/7 from a machine in a house, with scheduled shows, locally submitted commercials, station bumpers, a nightly flagship (**GROUND ZERO** – alien street interviews conducted via baguette), and a midnight sign-off to colour bars, exactly as civi-

lization intended.

This repository is the whole station: the playout configuration, the uplink, the tower, the storefront website, and the operating manual.

SEC.01

HOW IT WORKS



Exactly one stream ever leaves the house. The VPS does the crowd work, so a residential connection can run a channel with an unlimited audience. Latency is ~20 seconds and that is fine, because this is television, not a phone call.

SEC.02

REPOSITORY MAP

PATH	WHAT IT IS
docs/pdf/docs-build-guide.pdf	Start here. The full station build, Phase 0 → on-air
docs/pdf/docs-programming.pdf	The broadcast week. The full schedule, the show bible, and the ErsatzTV mapping

Continued on next page

(Continued)

PATH	WHAT IT IS
docs/pdf/docs-archive-fetch.pdf	Stocking the library. Public-domain intake from the Internet Archive
docs/pdf/docs-clustering.pdf	More than one machine. Node roles, failover, and what can't be load-balanced
docs/pdf/docs-self-hosting.pdf	Run master control on a homelab – Proxmox & TrueNAS, GPU passthrough, tips
docs/pdf/docs-gear-and-costs.pdf	Hardware picks, VPS comparison, bandwidth math, budgets
docs/pdf/docs-weather.pdf	On-air graphics. The bug, the weather desk, the crawl, and the traps in ErsatzTV's graphics engine
docs/pdf/docs-ad-standards.pdf	The one-page rulebook for locally submitted commercials
docs/pdf/docs-ideas.pdf	The writers' room – what's shipped, what's next
docs/comment-relay.md	The comment relay. How a viewer's comment gets from the storefront into the chat room: shape, allowance, house rules, post
site/index.html	The storefront – tabbed sections pinned to the window, live player (pause, volume, full screen), the tower's live chat in the rail, program grid, ad submissions (Riposte Labs design language)
site/retro/index.html	The original CRT-and-wood-cabinet version, preserved
site/_headers · site/_redirects	Cloudflare headers + short links (/watch, /lab)
vps/	The tower: Owncast docker-compose.yml + Caddyfile (and vps/peertube/ – a peer-to-peer alternative tower)
playout/	Master control: compose.yml (containerized), ErsatzTV launcher, uplink supervisor, now-playing bridge
tools/	Station scripts – see below
worker/	The storefront Worker – the moderated comment relay and nothing else (manual)
wrangler.jsonc	Cloudflare Worker config (serves site/ as static assets at ch0.ripostelabs.xyz, plus the relay)

Continued on next page

(Continued)

PATH	WHAT IT IS
<code>.env.example</code>	Domains, stream key, media root, tokens – copy, fill, never commit

STATION SCRIPTS

SCRIPT	JOB
<code>tools/bootstrap-node.sh</code>	Stand up a node from a bare machine – playout, tower, or worker (manual)
<code>tools/make-media-tree.sh</code>	Create the media library layout on the playout PC
<code>tools/z0-weather.sh</code>	The weather desk – renders the corner card, the crawl and the forecast segment (manual)
<code>tools/wire-graphics.py</code>	Wires the graphics elements and forecast slots into the broadcast week (manual)
<code>tools/fetch-archive.sh</code>	Stock the library from archive.org – public domain only, resumable (manual)
<code>tools/make-proxies.sh</code>	Build a test library – tiny stand-ins with identical runtimes, for trying schedules
<code>tools/test-failover.sh</code>	Prove the cluster fails over and never double-publishes
<code>tools/test-generators.sh</code>	Prove the card generators still make cards – arguments, text, 1080p/30 conformance
<code>tools/test-broadcast.sh</code>	Fire a live test pattern at the tower (build guide, Phase 1.5)
<code>tools/test-player-controls.mjs</code>	Drive the storefront player in a real browser – pause/resume-at-live, mute + volume, full screen (needs the channel on air)
<code>tools/test-section-tabs.mjs</code>	Drive the storefront's section tabs in a real browser – one panel at a time, deep links, keyboard, and that nothing spills past the fold

Continued on next page

(Continued)

SCRIPT	JOB
tools/test-rights-flag.mjs	Drive SEC.05 in a real browser – the provenance copy, and that the flag link carries the programme and a stamped moment into the mailto, tower up or down
tools/test-chat-panel.mjs	Drive the rail's live chat in a real browser – the SPEC/CHAT switch, the wire to the tower, and the sanitiser against hostile message bodies
tools/test-chat-composer.mjs	Drive the comment box in a real browser against a stubbed relay – every way it can say yes, no, and wait
tools/test-comment-relay.mjs	Drive the relay Worker with no browser and no network – shape, rate limits, fail-closed moderation, the tower call
tools/check-ad.sh	Screen a submitted spot: length, codecs, true loudness (read-only)
tools/normalize-ad.sh	Clear a submitted spot for air: 1080p/30, loudness-normalized
tools/make-colorbars.sh	Generate the midnight sign-off bars (with optional silence)
tools/make-testcard.sh	Generate the station test card – the branded signal-check pattern with a 1 kHz line-up tone
tools/make-signoff.sh	Generate the nightly sign-off – the "broadcast day concludes" card dissolving into colour bars
tools/make-ident.sh	Generate station idents – the "NOW WATCHING CHANNEL Z0" bumpers
tools/make-slate.sh	Generate slate cards – technical difficulties, sign-off, please stand by
tools/make-bug.sh	Generate the channel bug (watermark PNG) for ErsatzTV

SEC.03

QUICKSTART

The short version – the `build guide` has every command.

1. **Phase 0 – names.** A domain, a VPS, two DNS records. See `gear-and-costs` for picking the VPS (spoiler: it can be \$0).
2. **Phase 1 – the tower.** On the VPS: `vps/docker-compose.yml` up, Caddy configured, then immediately change Owncast's default admin password and stream key. Prove the pipe with `tools/test-broadcast.sh`.
3. **Phase 2 – master control.** On the playout PC (or a homelab box – see `self-hosting`): `tools/make-media-tree.sh`, fill the library (`tools/fetch-archive.sh --all` stocks the public-domain half – see `archive-fetch`), then either `playout/ersatztv.sh` or the whole containerized stack with `cd playout && docker compose up -d`. Build the schedule and ad-break filler in the ErsatzTV UI.
4. **Phase 3 – the uplink.** The `uplink` service in `playout/compose.yml` carries one stream to the tower and reconnects forever (or install the bare-metal `playout/z0-uplink.service`). Optionally add the now-playing bridge: `docker compose --profile nowplaying up -d`. You now run a television station.
5. **Phase 4 – the storefront.** Edit the `WATCH_HOST` + `AD_EMAIL` lines at the top of `site/index.html`. Cloudflare serves it via **Workers Static Assets**, wired to this repo through Cloudflare's Git integration (`wrangler.jsonc` points `assets` at `site/`) – every push to `main` auto-deploys. Live at `ch0.ripostelabs.xyz`.
6. **Phase 5 – go live sometimes.** OBS to the same RTMP key for Ground Zero remotes and Lab Hour.
7. **Phase 6 – more machines (optional).** `tools/bootstrap-node.sh` stands up a node from a bare box in one command, and a second playout node on the same shared media becomes a standby that takes over if the first dies. Note what this does and doesn't buy you – a linear channel can't be load-balanced, only made redundant. See `clustering`.

SEC.04

CONFIGURATION, ALL OF IT

- ▶ **.env** (from `.env.example`) – domains, stream key, channel URL, media root, and (optional) the Owncast token for the now-playing bridge. Used by the compose stack, the uplink, and the tools.
- ▶ **site/index.html CONFIG block** – stream URL, Owncast base (for the ON AIR light, receiver count, and the live NOW SHOWING title), ad-submission email, chat and lab links. Also the **P2P_* keys** (WebRTC segment sharing, on by default – viewers offload the tower for each other), the **CHAT_* keys** (see below) and **PEERTUBE_EMBED** (set it to run the PeerTube tower's player instead; see `vps/peertube/`).

THE LIVE CHAT IN THE RAIL

SEC.01's right-hand rail carries two things behind one switch in the card's head bar: the **SPEC** card and the tower's **CHAT**. It opens on CHAT, and the viewer's choice is remembered. They share the column rather than stacking because there is only ~16px of slack under the SPEC card on a 1280×900 window – measured, not guessed.

The chat is **read-only, on purpose**. The storefront reads the room straight from Owncast (`/api/chat` for the backlog, then a websocket) and posts nothing. A page that could post would have to carry a chat token, and a token in a public page belongs to everyone who views it. The way in for viewers is the tower itself, linked at the foot of the panel.

The way in is the relay – `worker/`, documented in `docs/comment-relay.md`. A viewer's comment goes to a Worker route on this origin, which checks its shape, spends the viewer's allowance, puts it in front of a small model against the house rules, and only then speaks to Owncast with a server-side token. Failure is closed: if the moderator cannot be reached, nothing is posted.

Until `OWNCAST_TOKEN` and a moderator are configured the relay reports itself closed, the comment box never appears, and the panel keeps its link to the tower. A storefront deployed without those secrets looks exactly as it did before the relay existed.

Three things about the tower are worth knowing before touching any of it, all of them found the hard way and all of them commented at the chat block in `site/index.html`:

- ▶ the register POST must be a **simple request** – send `Content-Type: application/json` and the preflight is refused by the tower's `Access-Control-Allow-Headers`, and the whole panel dies with "Failed to fetch";
- ▶ **USER_JOINED** is broadcast on a token's first connection only, so persisting the token in `localStorage` is what stops every page view announcing a new arrival in the room;
- ▶ message bodies are HTML the tower rendered from someone else's markdown, and this page sets no CSP, so they go through an allowlist and nothing else.
- ▶ **Everything else** lives in the ErsatzTV and Owncast admin UIs, documented in the build guide.

The relay's own two secrets live in Cloudflare, set with `wrangler secret put` and never in the repo: **OWNCAST_TOKEN** (an Owncast token scoped `CAN_SEND_MESSAGES` – its **name** is the author the chat room sees) and **ANTHROPIC_API_KEY** (skip it for `MODERATION_PROVIDER=workers-ai` or `off`).

Two more secrets, both in `.env` (gitignored) and nowhere else: the **stream key** (the world vs. your airwaves) and, if you run the marquee, an **Owncast access token**. Cloudflare needs no secret in the repo – its Git integration deploys the storefront on every push on its own.

SEC.05

THE SPONSOR

Channel Z0 is sponsored by **Riposte Laboratories** (ripostelabs.xyz) – a design company that transforms discarded plastic and end-of-life battery cells into durable, modular products. Between programs, you'll see what the lab has been building lately. The storefront speaks the lab's design language: paper, ink, one red, spec sheets, and esh's checkers.

PARRY • RIPOSTE • RECYCLE • REPEAT ♻

SEC.06

LICENSE

Code and configuration are MIT. The **Channel Z0** and **Ground Zero** names, show concepts, and the Riposte Laboratories marks and brand language are not granted by the licence – build your own station with this plumbing, but make it yours. (The Prelinger Archive material the guide points at is public domain; anything you air still has to be yours to air. See [docs/pdf/docs-ad-standards.pdf](#).)

This station concludes its broadcast day at midnight. Colour bars until sunrise.

SEC.07

BRAND

This project follows the Riposte Laboratories design system – published at [ri-postelabs.xyz/brand](#). See [docs/pdf/brand.pdf](#) for what conforms, what deliberately diverges, and why.

DOC NO. RL-Z0-AREV. A • EST. 2026 PARRY • RIPOSTE • RECYCLE • REPEAT
Riposte Laboratories Inc.

SOURCE README.md SHA fb99b64ed4fb BUILT 2026-09-07